



RAPPORT DE STAGE

Conception, développement et déploiement du nouveau site de la MJC de Mauguio-Carnon avec Django et Wagtail

Théodore DE BOISSESON

Tuteur de stage : Malo GASQUET

Maître de stage : Dominique GOURMELON

Établissement : Institut Universitaire Technologique de Montpellier-Sète

Structure d'accueil : Maison des Jeunes et de la Culture de Mauguio-Carnon

Année universitaire 2024 – 2025

Du 21 Janvier au 4 Avril

Remerciements

Avant tout, je tiens à remercier la Maison des Jeunes et la Culture de Mauguio-Carnon dans son ensemble, pour m'avoir accueilli pendant ces 11 semaines, ainsi que m'avoir permis de m'investir dans un projet enrichissant et pertinent.

Je remercie Dominique Gourmelon, qui m'a supervisé tout au long du stage, et toute l'équipe qui a été bienveillante, coopérative et pleine de bon conseils vis-à-vis de mon travail.

Je remercie aussi les administrateurs de l'association, qui m'ont accompagné au fil des commissions organisées, m'accompagnant tout au long du stage, me donnant un fil conducteur.

Enfin, je souhaite exprimer ma gratitude à l'ensemble des personnes externes à la structure avec qui j'ai échangé. Ils ont pu m'apporter une aide technique lorsque j'en avais besoin, notamment Xavier Astor, développeur de l'ancien site.

Résumé en français

Ce rapport présente mon stage de 11 semaines effectué à la Maison des Jeunes et de la Culture (MJC) de Mauguio-Carnon, dans le cadre de ma deuxième année de BUT Informatique. La mission principale consistait à moderniser le site web de l'association, dont l'ancienne version présentait des limitations techniques importantes et une ergonomie peu intuitive.

J'ai conçu et développé une nouvelle plateforme web en utilisant Django et Wagtail, un CMS basé sur ce framework. Cette solution offre une interface d'administration intuitive pour l'équipe de la MJC, tout en garantissant un site responsive et moderne pour les utilisateurs. Le projet a été mené de manière autonome, de l'analyse des besoins jusqu'au déploiement en production et à la formation des administrateurs.

Ce stage m'a permis d'approfondir mes compétences en développement web, notamment avec l'utilisation de technologies comme Django, Wagtail, PostgreSQL et TailwindCSS. J'ai également acquis une expérience significative en gestion de projet et en déploiement d'applications web, en configurant un environnement de production sécurisé sur un VPS.

Le résultat final est un site web moderne, accessible et facilement administrable, qui répond aux besoins spécifiques de l'association et améliore sa présence numérique.

Mots-clés : MJC Mauguio-Carnon, Django, Wagtail CMS, Python, Développement web, PostgreSQL, Responsive Design, TailwindCSS, VPS, Déploiement, Association culturelle, Interface d'administration, Git, CI-CD

Sommaire

Remerciements.....	2
Résumé en français.....	3
Sommaire.....	4
Table des figures.....	5
Glossaire.....	6
1. Introduction.....	7
2. Contexte du stage.....	8
2.1. Présentation de la structure d'accueil.....	8
2.2. Public ciblé.....	8
2.3. Enjeux du stage.....	9
3. Analyses.....	10
3.1. Analyse du sujet.....	10
3.2. Analyse de l'Existant.....	11
3.3. Analyse de l'Environnement Technique.....	14
Architecture Django et Wagtail.....	16
4. Rapport technique.....	17
4.1. Analyse technique et apprentissage préliminaire.....	17
4.1.1. Formation et montée en compétences.....	17
4.1.2. Analyse des besoins techniques spécifiques.....	17
4.1.3. Mise en place de l'environnement de développement.....	18
4.2. Cœur du développement.....	18
4.2.1. Conception des modèles Django et Wagtail.....	18
4.2.2. Structure des templates et intégration de Tailwind CSS.....	22
4.3. Formation, déploiement et affinage.....	24
4.3.1. Configuration du serveur de production.....	24
4.3.2. Optimisation des performances.....	25
4.3.3. Formation des administrateurs.....	25
4.3.4. Sécurité et maintenance.....	26
Conclusion de la partie technique.....	26
5. Méthodologie et Organisation du Projet.....	27
5.1. Applications et logiciels.....	27
5.2. Communication et réunions.....	32
5.3. Méthodologie de développement.....	32
6. Conclusion.....	33
7. Bibliographie.....	34

Table des figures

Figure 1 – Diagramme des cas d'utilisation du site web.....	12
Figure 2 – Page d'accueil de l'interface utilisateur du site précédent.....	14
Figure 3 – Page d'accueil de l'interface administrateur du site précédent.....	14
Figure 4 – Schéma de l'architecture technique du site en production.....	17
Figure 5 – Code du modèle Wagtail de la page activité.....	22
Figure 6 – Exemple de rendu d'une page d'activité minime.....	23
Figure 7 – Extrait de code du header responsive du site, montrant l'utilisation de TailwindCSS.....	24
Figure 8 – Rendu du header version téléphone.....	25
Figure 9 – Configuration de Nginx (reverse proxy) sur le VPS d'Oracle.....	26
Figure 10 – Capture d'écran de mon espace de travail "Stage" sur Notion.....	28
Figure 11 – Capture d'écran de ma page "Apprentissage préliminaire" sur Notion...	29
Figure 12 – Capture d'écran de ma page "Carnet de bord" sur Notion.....	30
Figure 13 - Capture d'écran détaillée de mon carnet de bord sur Notion.....	31
Figure 14 - Exemple d'une Pull Request du projet sur Github.....	32

Glossaire

MJC : Maison des Jeunes et de la Culture

Python : Un langage de programmation de haut niveau à usage général. Il dispose d'une bibliothèque standard étendue et d'un grand nombre de bibliothèques tierces, ce qui en fait l'un des langages les plus populaires pour l'apprentissage automatique, le traitement du langage naturel et les applications de science des données. [1]

CMS (Content Management System) : Un système de gestion de contenu, qui permet aux utilisateurs de créer, gérer et modifier le contenu d'un site web sans avoir besoin de compétences techniques avancées.

Django : Un framework web **Python*** utilisé pour développer des applications web. Il facilite la création de sites complexes et est connu pour sa sécurité, sa flexibilité et sa rapidité.

PostgreSQL : Une base de données relationnelle open-source utilisée pour stocker les données du site, offrant des fonctionnalités avancées comme la gestion de transactions et la réplication des données.

Responsive (Design) : Une approche de conception web visant à créer des sites qui s'adaptent automatiquement à la taille de l'écran de l'utilisateur, que ce soit sur un ordinateur, une tablette ou un smartphone.

SEO (Search Engine Optimization) : L'optimisation pour les moteurs de recherche, qui consiste à améliorer la visibilité d'un site web sur les moteurs de recherche comme Google.

Virtual Private Server (VPS) : Un serveur privé virtuel est un environnement isolé, créé sur un serveur physique à partir d'une technologie de virtualisation. Cette solution offre tous les avantages d'un serveur standard, profitant de ressources allouées et d'une administration complète. Vous pouvez choisir le système d'exploitation ainsi que les applications que vous souhaitez utiliser, sans contrainte de configuration. [2]

1. Introduction

Dans le cadre de ma deuxième année de BUT Informatique, j'effectue un stage de 11 semaines au sein de la **Maison des Jeunes et de la Culture (MJC*) de Mauguio-Carnon**. Ce stage me permet d'appliquer et d'approfondir mes compétences en développement web tout en répondant à un besoin concret de modernisation du site internet de l'association.

La MJC, en tant qu'acteur local, propose un large éventail d'activités culturelles, sportives et éducatives à destination des habitants. Cependant, son site actuel présente des limitations techniques et une ergonomie peu intuitive, rendant la gestion des contenus difficiles pour les équipes administratives.

L'objectif principal de ce stage est de **concevoir et de déployer un nouveau site web** plus moderne, accessible et facilement administrable. Pour cela, je développe une solution basée sur **Django*** et **Wagtail***, en tenant compte des attentes des utilisateurs finaux ainsi que des **besoins spécifiques de l'équipe** en charge de la gestion du site.

Ce rapport propose une analyse détaillée de l'existant, des problématiques rencontrées et des solutions techniques mises en œuvre pour répondre aux enjeux du projet.

Note importante : Ce rapport de stage a été finalisé le 27 mars en raison de la date de rendu imposée, qui précède la fin de la période de stage prévue le 4 avril. Il rend compte de ce qui a été effectué jusqu'à la 10e semaine de stage.

2. Contexte du stage

2.1. Présentation de la structure d'accueil

La Maison des Jeunes et de la Culture (MJC) de Mauguio-Carnon est une association loi 1901 à but non lucratif. Elle a pour mission principale de proposer des activités culturelles, artistiques et sportives pour un public varié, allant des enfants aux personnes âgées. Elle dispose de deux sites : un principal situé à Mauguio et un second à Carnon. Cette structure locale joue un rôle essentiel dans la vie socioculturelle de la commune en facilitant l'accès à des loisirs à des tarifs abordables. Étant une association, la MJC s'inscrit dans une dynamique sociale plutôt que commerciale.

L'association fonctionne grâce à une équipe de salariés restreinte, complétée par des bénévoles et des intervenants extérieurs. L'organisation est divisée en plusieurs pôles : administration, communication, animation et gestion des activités. J'ai travaillé en collaboration avec l'administration et les responsables de la communication pour la refonte du site web.

2.2. Public ciblé

La MJC s'adresse principalement aux habitants des communes de Mauguio et Carnon, ainsi qu'aux communes voisines. Son public cible est composé de familles, de jeunes et de seniors, cherchant à participer à des activités culturelles, sportives ou éducatives. Bien qu'il existe d'autres associations et structures municipales proposant des services similaires, la MJC se distingue par son engagement local et son offre diversifiée. L'enjeu est donc d'améliorer la communication numérique pour rendre ces services plus accessibles et visibles auprès du public.

2.3. Enjeux du stage

Le site web existant était obsolète, peu ergonomique et difficile à mettre à jour pour l'équipe administrative. Cette situation complique la diffusion d'informations sur les activités et les événements proposés pour les adhérents et visiteurs de la MJC. Mon rôle lors de ce stage était de concevoir une nouvelle version du site web, moderne et accessible, qui répond aux besoins de l'association et de ses utilisateurs.

L'un des principaux objectifs était de fournir un site **responsive***, capable de s'adapter aux différents supports (ordinateurs, tablettes, smartphones). Il doit également être facilement administrable par des personnes ne possédant pas de compétences techniques, garantissant ainsi une mise à jour régulière des informations. Pour cela, il était nécessaire de choisir un système de gestion de contenu (**CMS***) adapté.

Bien que WordPress soit le CMS le plus populaire au niveau mondial, j'ai opté pour une solution basée sur Django et Wagtail. Ce choix répond aux exigences pédagogiques de mon BUT Informatique, en me permettant de renforcer mes compétences en développement tout en proposant une solution sur mesure aux besoins de ce projet. Cette approche m'engage également à assurer la livraison d'une documentation claire, pour faciliter la prise en main du site par l'équipe administrative.

Mes responsabilités durant ce stage incluent :

- L'analyse du site existant et l'identification des principales lacunes.
- La conception et la réalisation d'un nouveau site web sur mesure.
- L'intégration d'une interface d'administration intuitive avec Wagtail.
- La rédaction d'une documentation technique pour faciliter la maintenance future.
- La formation de l'équipe administrative à l'utilisation du nouvel outil.

Ce projet répond à deux enjeux : moderniser l'image numérique de l'association et offrir une solution sur le long terme, simple d'utilisation pour l'administration du site.

3. Analyses

3.1. Analyse du sujet

La refonte du site web de la MJC répond à un besoin concret de modernisation et d'amélioration de la gestion du contenu. L'objectif principal est de créer une plateforme intuitive et visuelle pour les visiteurs cherchant des informations sur les activités et les événements. Cette nouvelle version doit également permettre à l'équipe administrative d'ajouter du contenu facilement sans nécessiter de compétences techniques avancées.

En collaboration avec les responsables de la communication et l'administration, j'ai défini les attentes principales :

- Un design moderne et responsive pour une expérience adaptée à tous les supports d'appareils accédant au site.
- Une interface d'administration facile avec un accès structuré au contenu et à l'édition.
- La possibilité d'ajouter, modifier et supprimer facilement divers types de pages.
- Une solution technique évolutive, compatible avec les futurs besoins de l'association.

Vous trouverez plus bas le diagramme des cas d'utilisation pour les Visiteurs, Administrateurs et Éditeurs du site.

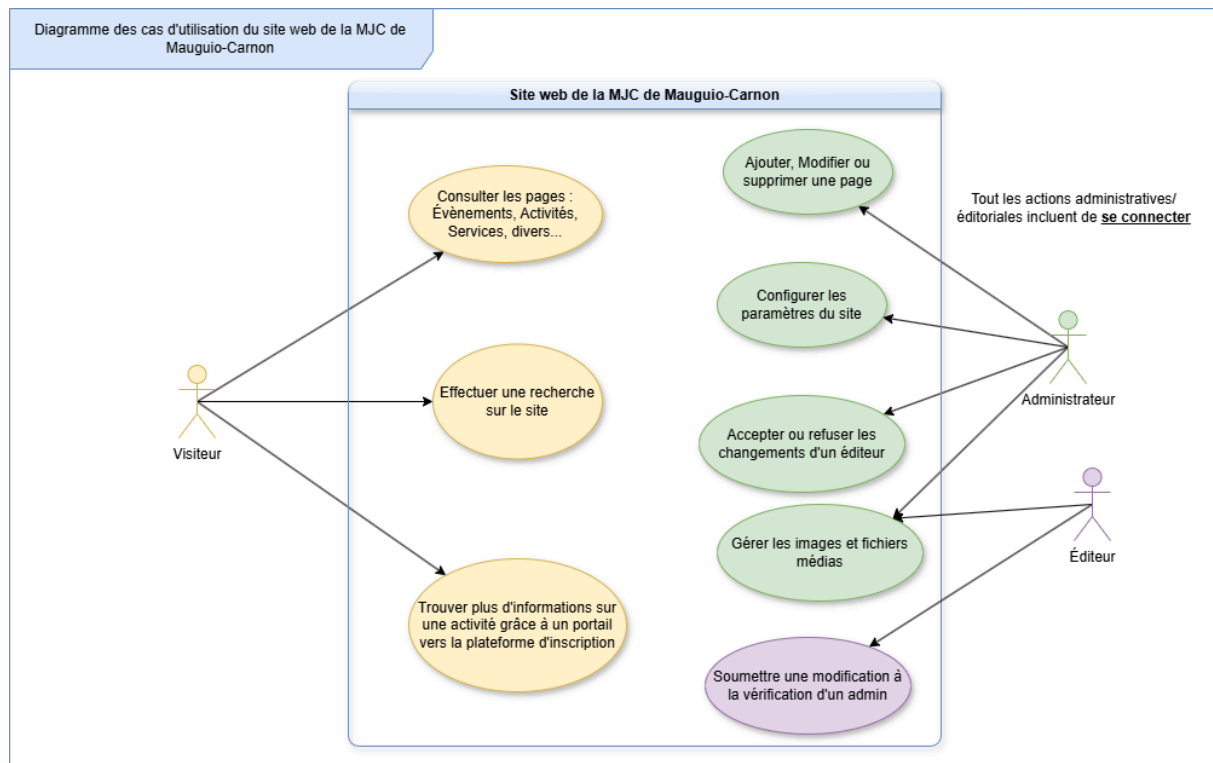


Figure 1 : Diagramme des cas d'utilisation du site web

3.2. Analyse de l'Existant

Lors de mon arrivée au début du stage, l'équipe de la MJC m'a fourni une documentation sur l'ancien site. On pouvait y retrouver diverses informations sur l'architecture technique... La base de données, l'hébergement, etc, mais surtout le lien internet vers le code source du projet, un dépôt sur Github. Avoir tous ces éléments à disposition m'a permis de bien comprendre l'existant et ses limites.

Le site web de la MJC présentait plusieurs limites qui nuisaient à son efficacité.

Voici les différents problèmes que j'ai identifié sur l'ancien site :

- **Ergonomie obsolète** : La navigation est peu intuitive, avec une structuration confuse des informations. Il en est de même pour l'interface administrative :
- **Maintenance complexe** : Les mises à jour du contenu nécessitent des interventions techniques en raison d'une interface mal structurée, non visuelle, rendant l'autonomie éditoriale difficile.

- **Non-adaptabilité** : Le site n'est pas entièrement responsive, ce qui complique l'accès aux informations sur les appareils mobiles.
- **Visibilité limitée** : L'absence d'une architecture optimisée nuit au référencement (**SEO***), réduisant la visibilité du site sur les moteurs de recherche.

L'ancien site web de la MJC souffrait également d'une dette technique importante. Le code source était peu structuré, mal documenté et utilisait surtout des technologies obsolètes. Cela étant, car l'ancien site a été codé en 3 semaines et surtout en 2017, donc il y a 8 ans. Le code étant en plus de cela non documenté (à la différence de l'architecture technique) et les dépendances trop anciennes. Par exemple Django utilisait la version 2.0.0, mais à présent Django est à la version 5.1.7. Cela rendait la maintenance complexe et coûteuse, et limitait les possibilités d'évolution du site.

Lorsque j'ai essayé par moi même de mettre à jour les dépendances sur l'ancien projet, il y avait des conflits de dépendances, d'autres qui n'étaient même plus supportées. Ca posait des problèmes de sécurité et de compatibilité.. En plus de cela je ne connaissais pas encore Django, c'est pourquoi j'ai décidé de repartir de zéro et avoir une base propre plutôt que de reprendre le code de l'ancien projet. Ce dernier a tout de même servi d'inspiration et d'exemple sur lequel m'appuyer.

Mon analyse de l'existant m'a permis d'identifier ces points critiques et d'orienter mes choix techniques pour répondre aux besoins concrets de l'association. Le nouveau site doit corriger ces faiblesses en offrant une interface plus moderne, ergonomique et facilement administrable. Mais notamment proposer un code évolutif et simple à prendre en main au développeur qui prendra la relève.

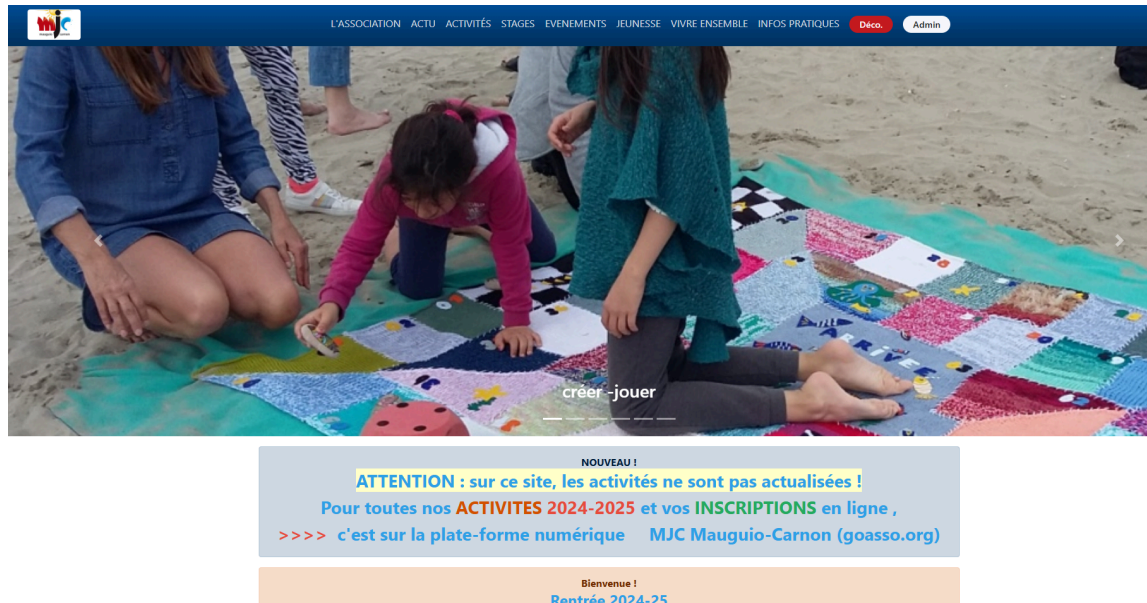


Figure 2 : Page d'accueil de l'interface **utilisateur** du site précédent

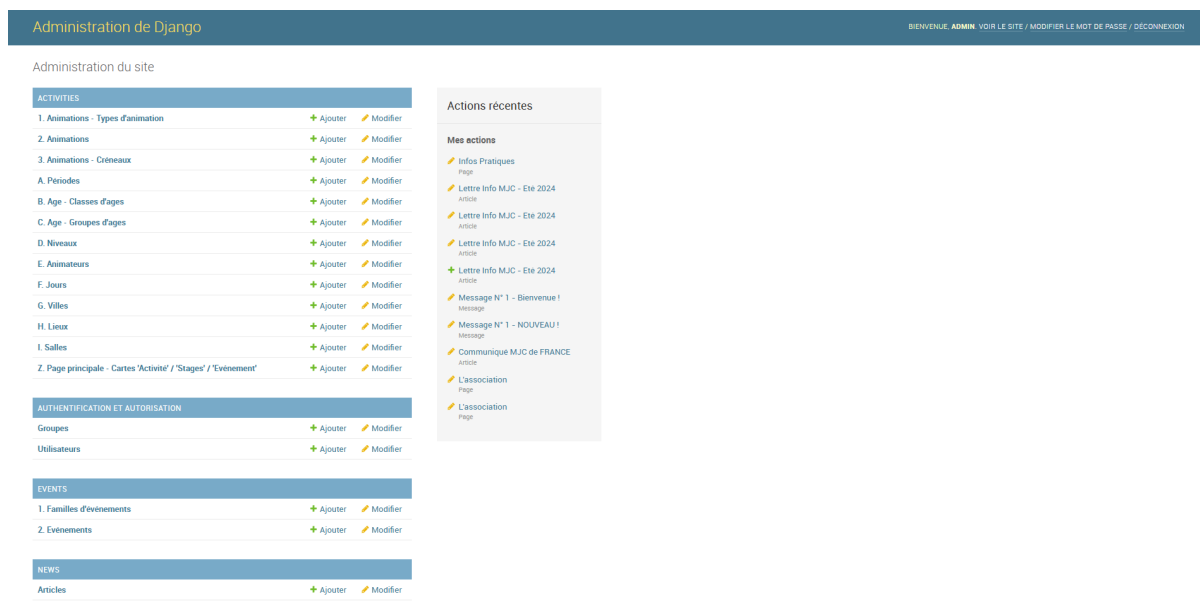


Figure 3 : Page d'accueil de l'interface **administrateur** du site précédent

3.3. Analyse de l'Environnement Technique

Le choix des technologies est crucial pour garantir la stabilité et la maintenance du site sur le long terme. Un mauvais choix dès le départ peut s'avérer fatal au milieu du projet. C'est pourquoi j'ai réservé la première semaine de mon stage exclusivement à de la recherche et de l'essai de différents environnements techniques. J'ai pu fixer un choix sur lequel j'avais confiance.

Voici l'environnement technique ressortissant :

- **Django** : Framework web Python reconnu pour sa robustesse, sa sécurité et sa structure modulaire. Il facilite la création de sites web complexes et personnalisables. C'est notamment ce même framework qui a été choisi par le développeur de l'ancien site, Xavier Astor.
- **Wagtail** : Un CMS basé sur Django, sert à améliorer ce dernier, offrant une interface utilisateur moderne et intuitive pour l'administration du contenu. Il permet d'adapter l'interface aux besoins spécifiques de la MJC. Contrairement à Wordpress (CMS le plus populaire et *No-Code*), il est nécessaire de coder pour le mettre en place et développer, mais il offre beaucoup plus de libertés. Son code source peut être *override* (surchargé), permettant de personnaliser et adapter tout le CMS pour avoir le résultat voulu, jusqu'à l'interface d'administration elle-même. Par exemple, j'ai ajouté un bloc suivi de l'espace disque restant sur le serveur, sur la page d'accueil de l'administration. Wagtail est un compromis entre rapidité et solution sur-mesure.
- **PostgreSQL*** : Base de données relationnelle open-source utilisée pour stocker les informations dynamiques du site. C'est-à-dire les informations modifiables grâce au CMS, tel que les titres, images et paragraphes. Django et Wagtail nécessitent une base de données pour leur fonctionnement et Postgres est celle recommandée à cet usage.
- **Tailwind CSS** : Framework CSS populaire et moderne facilitant la création rapide d'interfaces web esthétiques et responsives grâce à des classes css utilitaires.
- **Poetry** : Un gestionnaire de dépendances et un outil de gestion d'environnements virtuels pour Python, permettant de gérer facilement les bibliothèques et les versions de Python dans un projet.

Les points suivant ont été déterminés durant la phase de développement :

- **VPS* Free tier chez Oracle** : Un serveur virtuel privé proposé par la société *Oracle*, qui m'a permis de tester *gratuitement* mon site dans des conditions réelles en matière d'hébergement. Il m'a également permis de m'entraîner et d'apprendre à ma guise, sur les procédures de déploiement et d'intégration continue (CI/CD) avant de prendre la décision de prendre un abonnement VPS chez OVH avec les administrateurs de l'association de la MJC.
- **VPS chez OVH** : Destiné à l'hébergement et la mise en production finale du site, avec les capacités matérielles nécessaires pour un site stable. Je l'ai choisi comme serveur virtuel privé final car il a certains avantages :
 - **Sécurité renforcée** : Pare-feux intégrés et une meilleure gestion des attaques DDoS.
 - **Performance stable** : Moins de limitations CPU/RAM que le VPS gratuit d'Oracle, donc moins de risques de throttling (ralentissement forcé).
 - **Meilleure gestion des DNS et du nom de domaine** : Comme c'est la même plateforme qui vend le nom de domaine, tout est centralisé sur un même service.
- **Nom de domaine chez OVH** : Les noms de domaines sont ceux qui permettent à tous les liens d'internet d'avoir, à la place d'une adresse IP, composée uniquement de chiffres et peu mémorisable, une adresse simple composée de mots. E.g. : 142.251.33.174 a comme nom de domaine "google.com".
- **Gunicorn** (Green Unicorn) : Un serveur Http Python WSGI pour systèmes Unix (Linux), souvent utilisé pour exécuter des applications Django en production. Contrairement au serveur de développement de Django (runserver), il est conçu pour gérer des requêtes de manière performante et sécurisée.

Le choix de prendre l'hébergement et le nom de domaine sur la même plateforme permet une meilleure intégration et un meilleur support entre ces deux éléments. Il permet également de regrouper tous les services sur un même compte et une même facture, facilitant la gestion technique et administrative.

Cette architecture technique me permet de proposer une solution performante, évolutive et simple à prendre en main pour l'équipe non technique de la MJC.

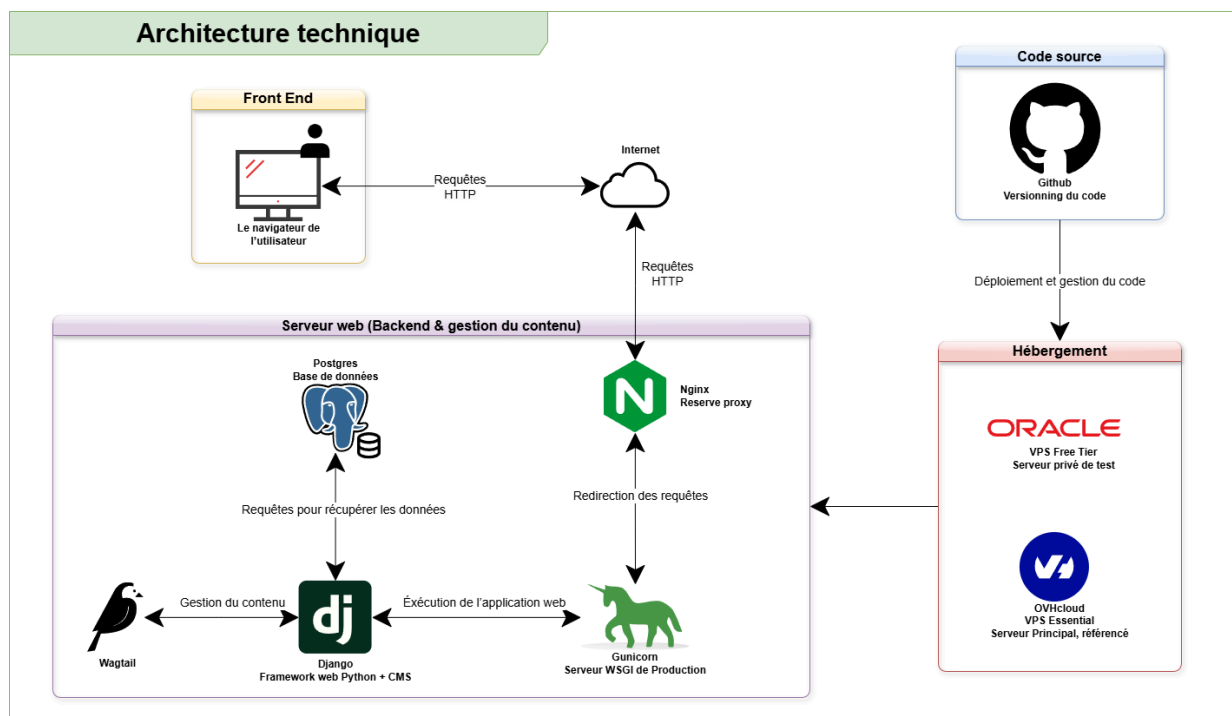


Figure 4 : Schéma de l'architecture technique du site en production

Architecture Django et Wagtail

Django suit une architecture **MTV (Model-Template-View)**, qui ressemble beaucoup au modèle MVC :

- **Model** : Gère la base de données et les règles métier.
- **Template** : Définit l'affichage des pages HTML.
- **View** : Récupère les données et les envoie aux templates.

Wagtail utilise un système de **pages hiérarchiques**, où chaque type de contenu est un modèle Django spécifique. Il utilise des **StreamFields**, qui permettent de créer du contenu dynamique avec différents blocs (texte, images, vidéos, etc.). Cette architecture est intuitive car similaire à celle des liens au niveau de la navigation des pages. Ce qui rend la compréhension plus facile pour un non-développeur.

Le CMS Wagtail se différencie de WordPress par sa flexibilité et sa capacité à être personnalisé en profondeur, tout en gardant une interface simple pour les administrateurs non techniques.

4. Rapport technique

Je connaissais déjà la MJC de Mauguio-Carnon et certains membres de son équipe, grâce à mon cercle social. Avant même que j'envisage un stage, il m'était déjà clair qu'il leur fallait un meilleur site web. C'est le seul et unique projet que j'ai traité lors de mon séjour.

Je vais vous présenter le rapport techniques en suivant les trois phases principales du projet :

- L'analyse technique et l'apprentissage préliminaire
- Coeur du développement
- Formation, déploiement et affinage

4.1. Analyse technique et apprentissage préliminaire

Cette première phase a été cruciale pour poser les fondements du projet. Après avoir analysé l'existant, j'ai dû me former aux outils qui allaient être employés pour la refonte du site.

4.1.1. Formation et montée en compétences

N'ayant jamais utilisé Wagtail auparavant, j'ai consacré 2 semaines à me familiariser avec ce CMS ainsi que sur Django[3]. J'ai suivi le tutoriel officiel de Dango ainsi que Wagtail pour créer un blog simple, ce qui m'a permis de comprendre:

- La structure de prof Django
- La configuration du serveur et les environnements de dev et prod
- La structure des modèles Wagtail (Pages et StreamFields)
- Le système de templates et la gestion des médias
- L'interface d'administration et ses possibilités de personnalisation

4.1.2. Analyse des besoins techniques spécifiques

Après avoir compris le fonctionnement de Wagtail, j'ai analysé les besoins

spécifiques de la MJC:

- Création d'un modèle de données adapté aux activités et événements de l'association
- Définition d'une architecture flexible permettant d'ajouter facilement de nouveaux types de contenus
- Réflexion sur l'optimisation du référencement (**SEO**)

4.1.3. Mise en place de l'environnement de développement

J'ai configuré un environnement de développement utilisant:

- **Poetry** pour la gestion des dépendances et environnements virtuels Python.[4]
- Git pour le versionnement du code
- Un serveur local pour les tests : runserver

4.2. Cœur du développement

Durant cette phase j'ai débuté et entamé la majorité du développement des fonctionnalités principales du site.

4.2.1. Conception des modèles Django et Wagtail

Les modèles ont été retravaillés de nombreuses fois, que ce soit au niveau visuel ou des différents champs modifiables sur une page. Après donc plusieurs itérations, j'ai conclu que plusieurs modèles se ressemblaient, ce qui a donné lieu à une encapsulation de certains des modèles de page. Il y a 10 modèles différents au total, mais ce nombre peut évoluer en fonction de l'affinage que je ferais après la rédaction de ce rapport.

Je vais donc vous lister les modèles principaux :

- **Page d'accueil** : Son nom est assez descriptif, la page comporte une section hero, un paragraphe optionnel dans le cas où l'on veut faire passer un message spécial et enfin une section d'aperçu des prochains événements de la MJC. On peut y modifier :

- Le titre d'introduction au site
- Les mots à défiler (ou rotatifs) qui permettent de montrer dynamiquement ce qu'il se fait à la MJC
- L'image de fond de la section Hero
- L'URL de la vidéo du moment, intégrée à la page
- Le paragraphe optionnel qui est un champ Richtext
- **Page Liste** : Affichant toutes les sous pages sous forme de cartes, avec :
 - L'image de la sous-page (si présente) en vignette
 - Un titre
 - Une description brève de la page qui prend le sous-titre et sinon c'est un extrait du contenu de la page.

La page contient également champ d'introduction optionnel pour présenter brièvement.

- **Page de détail** : Format générique de page, sur laquelle on peut présenter diverses informations. Elle contient les champs suivants :
 - Un titre et sous-titre
 - Le contenu : Un champ richtext
 - Une image, qui sert d'abord de visuel mais également afin d'avoir une vignette pour la page liste, souvent page parente.

Le reste des modèles, que je vais citer ci-dessous, sont des déclinaisons (ou héritages) des formats génériques **Page de liste** et **Page de détail**. Ces pages ajoutant des champs supplémentaires en fonction du besoin (et modifient des champs hérités dans de rares cas), ce sont les suivantes :

- **Liste de l'Association** : Déclinaison de la page liste, elle à un contenu rich text à la place de la description brève. Elle est vouée à fusionner avec la page liste.
- **Liste des activités** : En plus de la page liste, elle comporte aucun champ supplémentaire mais la template (vue), rajoute des boutons de tri (par nom/nouveauté) et de recherche.
- **Liste Agenda** : Même idée, elle ajoute du tri par date/nom, choix de la ville où se déroule l'évènement et enfin une barre de recherche. Elle affiche les

événements à venir

- **Évènements passés** : Similaire à la liste agenda mais affiche les événements passés.
- **Évènements** : Pages affichées dans les deux modèles ci-dessus, elle héritent de la page de détail et y ajoutent :
 - Un champ de date de début et fin de l'évènement
 - Un lieu
- **Page Association** : Déclinaison de la page de détail, elle ajoute le choix de décider d'afficher l'image sur la page ou qu'elle serve uniquement de vignette. Ce modèle est voué à devenir la nouvelle page de détail pour offrir plus de libertés avec un seul même modèle.
- **Activités** : Modèle expliqué en détail dans l'exemple plus bas dans cette partie.

Tous les modèles de pages héritent un moment ou un autre de la classe *Page* qui est le modèle générique proposé par Wagtail, qui comporte un titre. Donc toutes les pages ont au minimum un titre.

Pour les modèles d'activités, j'ai utilisé les StreamFields de Wagtail qui permettent aux administrateurs de construire du contenu dynamique avec différents types de blocs (texte, image, citation, etc.)[5].

```

@register_snippet
class Animateur(models.Model):
    name = models.CharField(max_length=255, unique=True)

    def __str__(self): ...

class SubActivity(Orderable):
    page = ParentalKey(*args: 'activites.ActivityPage', related_name='sub_activities', on_delete=models.CASCADE)
    title = models.CharField(max_length=255)
    description = RichTextField(blank=True)

    panels = [
        FieldPanel('title'),
        FieldPanel('description'),
    ]

    def __str__(self): ...

class ActivityPage(DetailPage):
    DetailPage.content.verbose_name = "Description"
    animateurs = ParentalManyToManyField(to: 'activites.Animateur', blank=True)
    link = models.URLField(
        max_length=200,
        help_text="Lien de redirection pour le bouton",
        blank=True,
    )

    content_panels = DetailPage.content_panels + [
        FieldPanel(field_name: 'animateurs', widget=CheckboxSelectMultiple),
        InlinePanel(relation_name: 'sub_activities', label="Sous Activités"),
        FieldPanel('link'),
    ]

class Meta: ...

```

Figure 5 : Code du modèle Wagtail de la page activité

J'ai choisi l'exemple ci-dessus des pages d'activités pour montrer la structure du code, car c'est un cas assez complet montrant divers aspects de Wagtail, sans être trop verbeux. La classe `Animateur` est un Snippet ou Fragment en français, elle permet dans une section indépendante de l'administration, d'ajouter et lister tous les animateurs de la structure. On peut ensuite les retrouver sur les pages d'activités.

Ensuite, on peut ajouter des sous activités, par exemple : "Anglais débutant, Anglais avancé" ou encore "Danses espagnoles enfant, Danse espagnoles adultes". C'est un ajout pratique aux activités, car il permet de regrouper synthétiquement plusieurs catégories d'une même activité, sur un même page. Ainsi facilitant la navigation pour l'utilisateur grâce à la quantité réduite de pages et des informations regroupées de manière intuitive. Un parent pourrait potentiellement voir qu'il peut inscrire, lui et son enfant à la même activité, mais de niveaux différents.

Enfin, parlons de la classe *page d'activité* elle-même. Elle hérite le modèle de page générique "DetailPage" et de ses champs (content_panels) puis à ça, elle y ajoute :

- Un champ pour préciser les animateurs supervisant l'activité
- Un champ pour le lien vers la plateforme d'inscription
- La classe des sous activités

Voici une capture d'écran pour mieux illustrer mes propos :

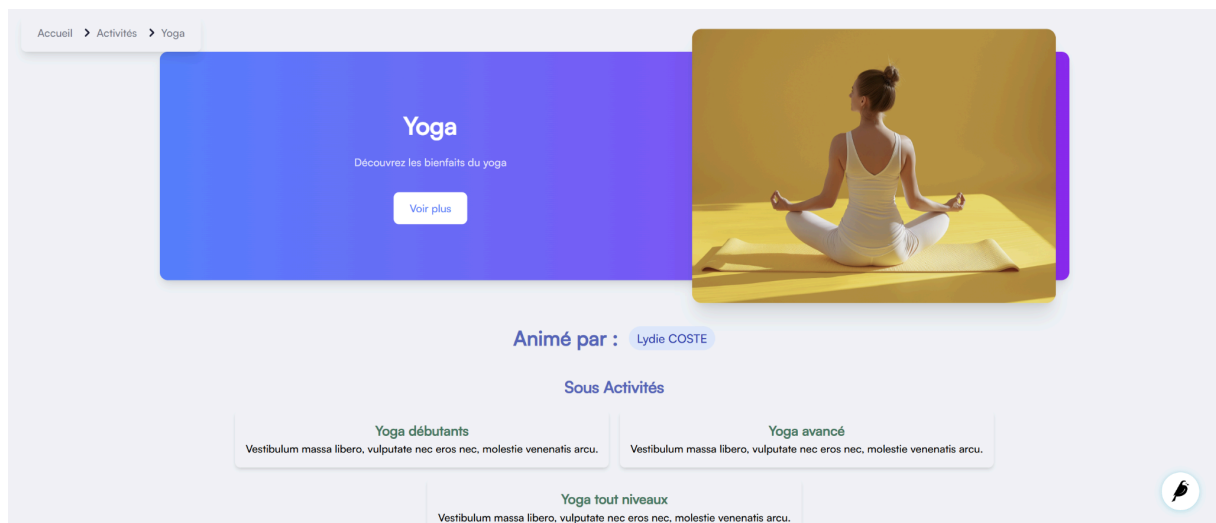


Figure 6 : Exemple de rendu d'une page d'activité minime

Je suis actuellement à la dernière phase de développement lors de la rédaction de ce stage. Donc attention, puisque je vais affiner certains détails, le site est susceptible d'éventuels changements dans la version finale.

4.2.2. Structure des templates et intégration de Tailwind CSS

J'ai créé une architecture de templates basée sur l'héritage, avec:

- Un template de base (base.html) contenant les éléments communs (header, footer, navigation)
- Des templates spécifiques pour chaque type de page, héritant de la template de base

Pour le design, j'ai utilisé TailwindCSS [6], un framework utilitaire qui m'a permis de créer une interface responsive sans avoir à écrire de CSS personnalisé, ce qui facilite la maintenance future. Voici un exemple :

```

{# Mobile Menu Content #}
<div class="mt-6 flow-root">
  <div class="-my-6 divide-y divide-gray-500/10">
    <div class="space-y-2 py-6 -mx-3">
      {# Search Button #}
      <a href="{% url 'search' %}" class="px-4! py-2! btn btn-primary flex items-center text-center gap-2">
        <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24"...>
        Rechercher
      </a>
      <a href="/" class="mobile-nav-link {% if request.path == '/' %}active{% endif %}">Accueil</a>
      {% for page in site_root.get_children.live %}
        <div class="flex w-full items-center justify-between rounded-lg mobile-nav-link">
          <a href="{{ page.url }}"
            class="py-2 pr-3.5 pl-3 {% if page.url == request.path %}active{% endif %}">
            {{ page.title }}
          </a>
          {% if page.get_children.live.in_menu.count > 0 %}
            <button class="flex flex-1 justify-end self-stretch items-center cursor-pointer"
              onclick="toggleSubMenu(this)">
              <svg class="size-5 transition-transform duration-300" viewBox="0 0 20 20"...>
            </button>
          {% endif %}
        </div>
        {% if page.get_children.live.in_menu.count > 0 %}
          <div class="submenu mt-2 space-y-2 overflow-hidden transition-all duration-300 max-h-0"...>
          {% endif %}
        {% endfor %}
      </div>
    </div>
  </div>
</div>

```

Figure 7 : Extrait de code du header responsive du site, montrant l'utilisation de TailwindCSS

Toutes les classes de la capture d'écran ci-dessus, sont des classes utilitaires de TailwindCSS. À l'exception de la classe `mobile-nav-link` (utilisant elle-même TailwindCSS), aucun style n'est écrit dans un fichier CSS externe. Voici son rendu ci-dessous :

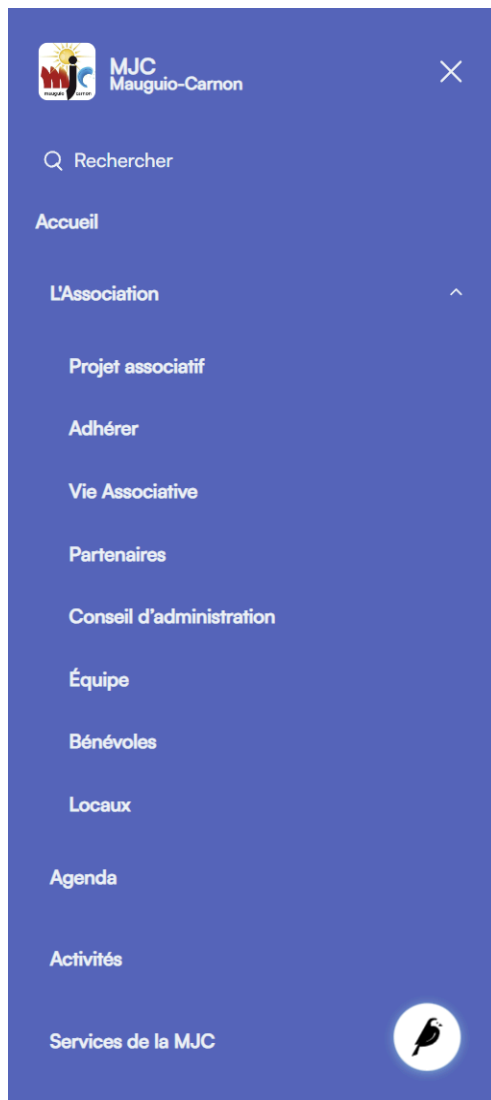


Figure 8 : Rendu du header version téléphone

Un autre aspect pratique de TailwindCSS, c'est ses variables de thème [7]. Par exemple, si on définit une couleur de thème, on obtient des classes utilitaires basées sur cette couleur. Permettant en changeant simplement la variable, de modifier d'une traite tout le thème du site. Dans l'extrait de code plus haut, j'utilise une classe `bg-primary`, couleur de fond qui se base sur la couleur primaire, ici définie sur du bleuet.

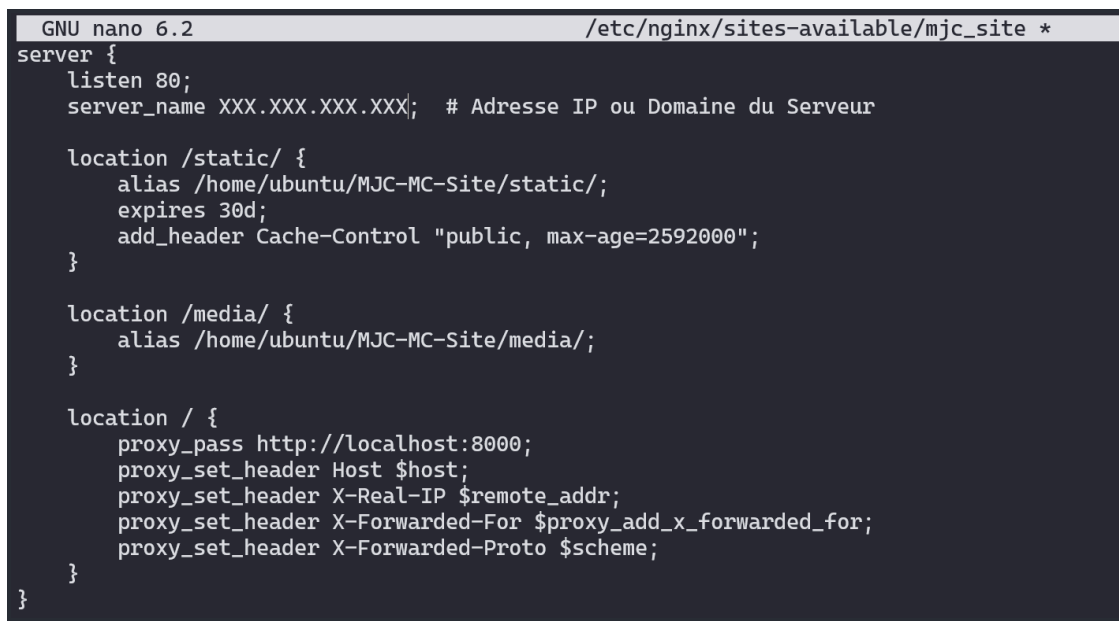
4.3. Formation, déploiement et affinage

4.3.1. Configuration du serveur de production

Pour le déploiement, j'ai configuré un **VPS** chez OVH[8] avec:

- **Nginx** comme serveur web frontal
- **Gunicorn** comme serveur d'application WSGI
- **PostgreSQL** comme base de données (comme en développement)[9]

Une des difficultés majeures a été la configuration correcte de Gunicorn derrière Nginx. J'ai rencontré des problèmes avec les fichiers statiques et medias qui n'étaient pas correctement servis. La solution a consisté à configurer correctement les chemins dans le fichier de configuration Nginx:



```
GNU nano 6.2 /etc/nginx/sites-available/mjc_site *
server {
    listen 80;
    server_name XXX.XXX.XXX.XXX; # Adresse IP ou Domaine du Serveur

    location /static/ {
        alias /home/ubuntu/MJC-MC-Site/static/;
        expires 30d;
        add_header Cache-Control "public, max-age=2592000";
    }

    location /media/ {
        alias /home/ubuntu/MJC-MC-Site/media/;
    }

    location / {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Figure 9 : Configuration de Nginx (reverse proxy) sur le VPS d'Oracle

4.3.2. Optimisation des performances

Pour améliorer les performances du site, j'ai mis en place:

- Un système de cache avec Django cache framework
- La compression des fichiers statiques avec White Noise
- Le préchargement des ressources critiques

4.3.3. Formation des administrateurs

La dernière phase a consisté à former l'équipe administrative à l'utilisation de Wagtail. J'ai pris le temps avec chacune des personnes qu'il allait utiliser l'interface administrative, pour les former individuellement. Cela prenant entre 30min et 1h par personne et me permettait d'ajuster mes mots à la personne que je formait.

J'ai également personnalisé l'interface d'administration Wagtail pour la rendre plus intuitive, en ajoutant des indications contextuelles et en simplifiant certains formulaires.

4.3.4. Sécurité et maintenance

Pour garantir la sécurité du site, j'ai:

- Configuré HTTPS avec Let's Encrypt
- Restreint les ports accessibles du VPS avec UFW
- Mis en place des sauvegardes automatiques de la base de données avec Crontab
- Créé un script de déploiement continu *deploy.sh*.

Conclusion de la partie technique

La refonte du site de la MJC a été un projet complet qui m'a permis d'appliquer mes connaissances en développement web tout en découvrant de nouveaux outils comme Wagtail. Les choix techniques effectués (Django/Wagtail au lieu de WordPress, Tailwind CSS pour le front-end) ont été guidés par la volonté de créer une solution sur mesure, évolutive et facile à maintenir sur le long terme. J'ai pu également approfondir mes compétences en communication et de travail en équipe en collaborant avec l'équipe administrative et les responsables de communication. Puis en les formant à l'utilisation de l'interface administrative, créée justement pour eux.

Le résultat est un site moderne, responsive et accessible, qui répond aux besoins spécifiques de l'association tout en simplifiant grandement la gestion du contenu.

5. Méthodologie et Organisation du Projet

J'ai eu beaucoup de libertés sur mes méthodes et mon organisation lors de ce projet, ce que j'ai donc défini entièrement par moi-même. Il est selon moi important de citer même les parties non techniques de mon organisation.

5.1. Applications et logiciels

J'ai organisé mon travail en utilisant le logiciel **Notion** pour la prise de notes, la planification des tâches et le suivi de l'avancement avec le carnet de bord. C'est un outil que j'utilise depuis plusieurs années, qui permet d'organiser ses notes sous forme de d'espace de travail. Il m'a aidé à regrouper toutes mes réflexions et décisions techniques.

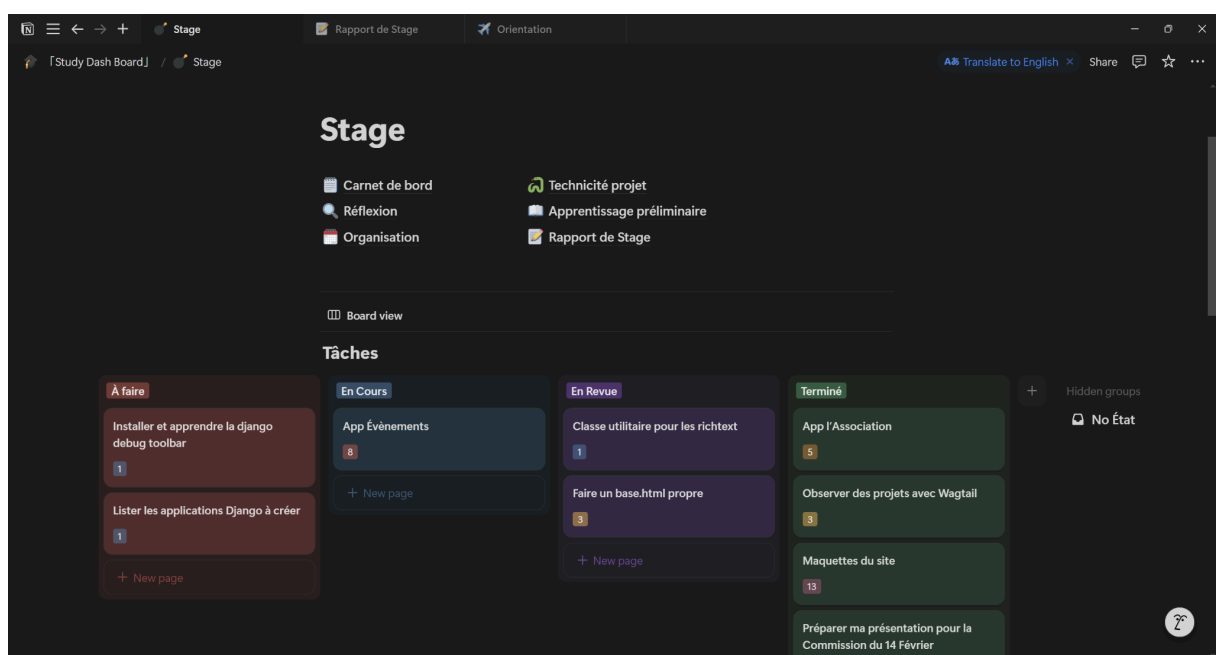


Figure 10 - Capture d'écran de mon espace de travail "Stage" sur Notion

La capture d'écran ci-dessus est la page principale de mon espace de travail du stage, sur le logiciel Notion. J'ai regroupé mes notes sous 6 pages différentes (Carnet de bord, Technicité projet, etc...). Et en bas de la page se trouve un Kanban

board, répliquant le tableau de gestion de projet Agile SCRUM, appris en cours précédemment cette année.

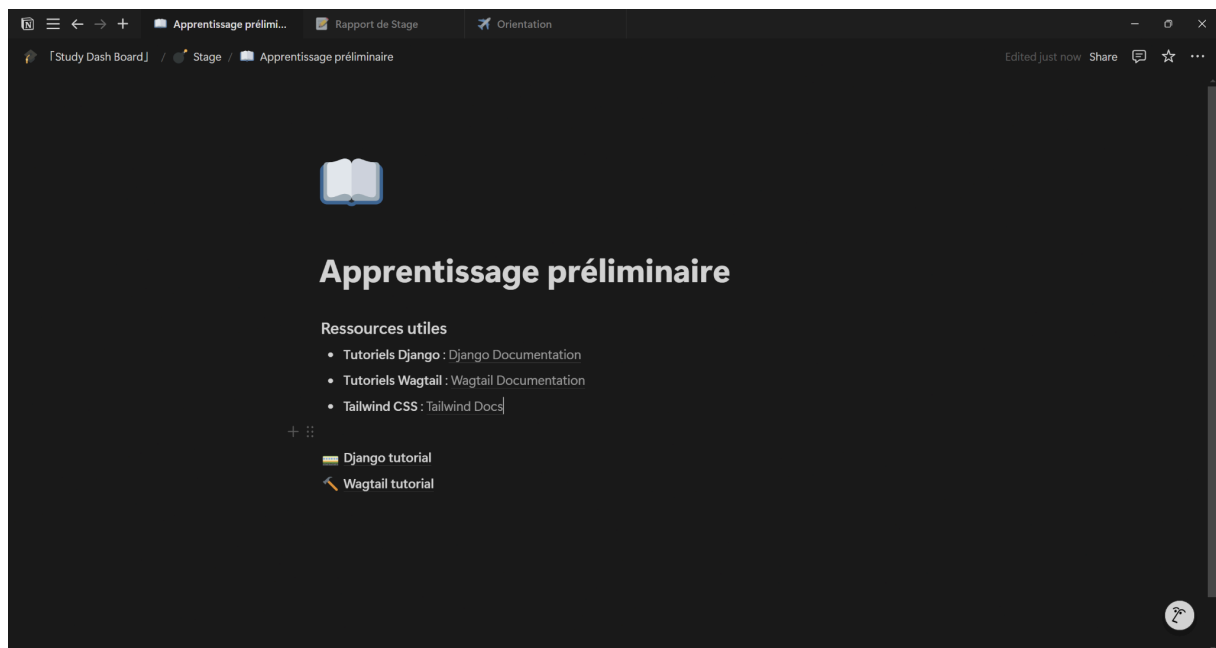


Figure 11 - Capture d'écran de ma page "Apprentissage préliminaire" sur Notion

En traitant dans l'ordre chronologique, cette page m'a permis de prendre des notes sur ce que j'avais appris lors de ma phase d'apprentissage au début du stage. Les ressources utiles, me permettant un accès rapide aux documentations des outils utilisés. Les deux pages Django et Wagtail tutorial étant respectivement les prises de notes faites lors des tutoriels.

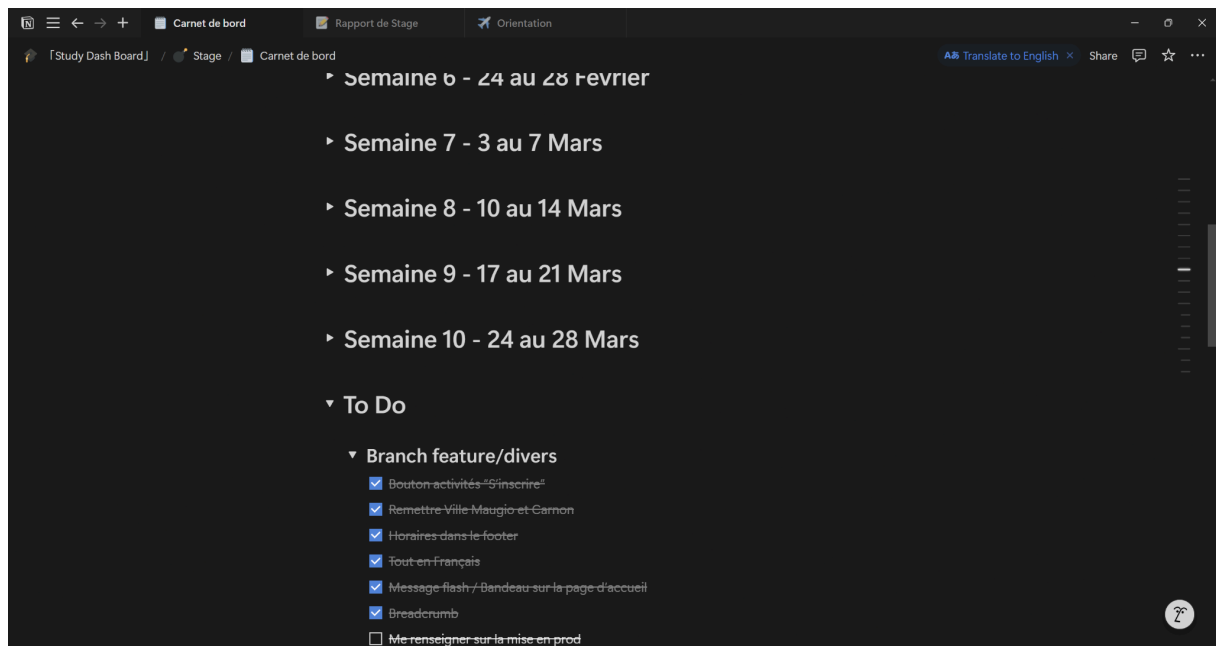


Figure 12 - Capture d'écran de ma page "Carnet de bord" sur Notion

J'ai également pensé à faire un carnet de bord, comme il était recommandé. J'y notais ce que j'avais fait chaque jour, ce que j'avais bien fait, ce que je pouvais améliorer, ainsi que mon ressenti de la journée, relativement au projet. Ces notes sont assez personnelles, c'est pourquoi je ne montre pas tout sur la capture d'écran.

Si vous n'êtes pas familier avec l'interface de Notion, il est bon de savoir que les triangles à gauche de chaque titre, est un bouton pour ouvrir le menu déroulant. C'est-à-dire que ligne de semaine est déroulable afin de montrer les notes faites chaque jour.



Figure 13 - Capture d'écran détaillée de mon carnet de bord sur Notion

On peut voir les menus déroulants (ou “toggle”) en action ci-dessus. C’est une fonctionnalité assez pratique puisqu’elle permet de réduire les parties sur lesquelles on ne porte pas notre attention sur le moment, permettant de se concentrer sur l’essentiel.

J’ai également utilisé **Google Agenda** pour organiser mon temps et prévoir les tâches importantes, noter les prochaines réunions.

Diagrams.net, anciennement Draw.io, est l’outil en ligne dont je me sers afin de faire les différents schémas de ce rapport.

Concernant la gestion du code, j’ai utilisé **GitHub** pour le versionnement du projet, en appliquant une structure de branches propre :

- Une branche main pour la version stable.
- Une branche dev pour les modifications en cours.
- Des branches spécifiques pour chaque lot de modifications (feature/*).

Même si j’étais le seul à travailler sur ce dépôt Github, j’ai bien fait en sorte savoir des descriptions pertinentes de mon travail, une organisation propre et des pratiques conventionnelles. C’est une pratique d’autant plus importante dans l’éventualité où

un développeur reprendra mon code dans le futur. J'ai par exemple pris le temps de décrire, passer en revue et squash chacune de mes **pull-requests**. J'ai pensé nécessaire de faire des pull request au lieu de directement fusionner les branches, afin de prendre le temps de vérifier et documenter mon travail.

Modifications diverses : Ergonomie, Customisation du site par les admins, Recherche et 404, Refactor et modeles génériques... #5

General :

- Site et Administration en Français
- [Ajout Breadcrumb navigation](#)
- [Ajout Flash Message](#) (modifiable dans les settings) fermé sur la session
- [Ajout Whitenoise](#)
- Footer : Ajout des horaires Carnon et rework le layout du footer + Liens : Plan du Site, Administration, mon Github
- [Ajout fonctionnalité de recherche élastique : Page et boutons dans le header](#)
- [Ajout Suivi du stockage serveur sur la page admin](#)
- [Page 404](#)
- [feat: Option réglage du Theme de couleurs depuis l'administration](#)

App Common :

- Ajout 'ContentMixin' pour facilement inclure des richtext aux modeles + refactor adapté
- [Modèles génériques de pages utilisables : DetailPage + nouveau BaseIndexPage](#)
- [Bouton retour en bas de toutes les pages detail et héritantes](#)

App Home :

- [Ajout section Info flash à la page d'accueil](#)

App Agenda :

- [Rajout de l'option "Mauguio et Carnon" pour les events](#)
- [fix: event end_date ne peut plus être inférieur à la start_date](#)
- [fix: Layout des event page pour affiche la date meme sans image](#)

Les quelques derniers commits sont des modifs faites sur le tas pour tenter de deployer le site sur Heroku

theodoredeboisseson added 30 commits 3 weeks ago

Rajout de l'option "Mauguio et Carnon" pour les events

923c715

Right sidebar:

- Reviewers:** No reviews
- Assignees:** No one — [assign yourself](#)
- Labels:** None yet
- Projects:** None yet
- Milestone:** No milestone
- Development:** Successfully merging this pull request may close these issues.
- Notifications:** Unsubscribe (You're receiving notifications because you're watching this repository.)
- 1 participant:** (Avatar)
- Lock conversation**

Figure 14 - Exemple d'une Pull Request du projet sur Github

Les lignes soulignées sont des liens vers les commits effectuant les changements décrits. J'utilisais les squash et les rebases pour les méthodes de fusions afin d'avoir un historique de "commits" vraiment linéaire et propre. Sachant qu'il est rare d'avoir des conflits de fusion (merge) lorsqu'on travaille sur un projet. Les revues de code et la gestion des versions ont été essentielles pour maintenir un code propre et bien organisé tout au long du développement.

5.2. Communication et réunions

Tout au long du stage, j'ai participé aux réunions hebdomadaires de la MJC, où je faisais un point d'avancement sur le site et recevais des retours des différents membres de l'équipe. J'ai également échangé régulièrement avec le secrétariat, futurs éditeurs du site, pour ajuster les besoins du site en fonction de leurs attentes. C'est-à-dire que je concevais l'application avec l'utilisateur.

En plus des réunions hebdomadaires, il y avait une commission "site web", dédiée entièrement à mon projet, qui a été fondée pour mon stage. Elle se composait des membres du Conseil d'Administration de la MJC. Chacunes de ces commissions étaient des jalons dans mes phases de développement. Me permettant, avec l'aide du CA, de fixer des deadlines pour avoir un planning clair des étapes du projet, et d'être accompagné lors de la prise de décisions importantes, tel que le choix de l'hébergeur ou encore la structure des pages du site.

5.3. Méthodologie de développement

J'ai suivi une approche **itérative** pour mener à bien ce projet, en travaillant par étapes successives, tout en restant sur une approche similaire à ce qui nous est enseigné à l'I.U.T. Voici les différentes étapes :

1. **Analyse et conception** : Étude des besoins, identification des fonctionnalités clés.
2. **Développement initial** : Mise en place de la structure Django et intégration de Wagtail.
3. **Tests et corrections** : Déploiement sur un environnement de test, correction des bugs.
4. **Mise en production** : Hébergement sur OVH, configuration de Unicorn et Nginx.
5. **Formation des administrateurs** : Explication de l'interface Wagtail et rédaction de la documentation d'utilisation.

Cette méthodologie m'a permis d'avancer de manière structurée et de garantir un produit fonctionnel et adapté aux besoins de la MJC.

6. Conclusion

Ce stage à la MJC de Mauguio-Carnon m'a permis de mener un projet de A à Z, en autonomie encadrée. J'ai pu concevoir et mettre en production un site web moderne répondant aux besoins de l'association, tout en développant mes compétences en **développement web, gestion de projet et déploiement d'applications**.

En termes d'apprentissage, ce stage m'a permis d'approfondir **Django et Wagtail**, d'apprendre à gérer un **serveur VPS** et à sécuriser un site en production. J'ai également acquis une méthodologie plus rigoureuse dans l'organisation de mon travail, notamment en utilisant Notion pour la planification et GitHub pour le versionnement du code.

Ce projet a eu un **impact direct sur la MJC** : l'équipe dispose maintenant d'un outil moderne et facile à administrer. Les administrateurs peuvent gérer les contenus de manière autonome, sans nécessiter de compétences techniques avancées.

Pistes d'amélioration et évolutions possibles :

- Améliorer l'accessibilité en respectant plus strictement les normes **WCAG**.
- Optimiser encore les performances du site avec un meilleur système de cache et un CDN comme AWS S3.
- Automatiser davantage le déploiement continu CI/CD pour simplifier les mises à jour futures.

En conclusion, ce stage a été une expérience très enrichissante, tant sur le plan technique que personnel. Il m'a donné l'occasion d'apprendre et de pratiquer de nombreux aspects de l'informatique qui ne m'ont pas encore été enseignés.

Signatures

Maître de stage

D. GOURMECON



Maison des Jeunes et de la Culture
527 av. du 8 Mai 1945 - 34130 MAUGUIO
Tél. 04 67 21 11 11 Fax 04 67 12 00 69
SIRET 478 176 0000 - APE 9493Z

L'étudiant

29/03/25

Inf

Bibliographie

- [1] « Tout savoir sur le langage de programmation Python », IA School.
[En ligne]. Disponible sur:
<https://www.intelligence-artificielle-school.com/langage-programmation/python/>
- [2] « Qu'est-ce qu'un VPS ? » [En ligne]. Disponible sur:
<https://www.ovhcloud.com/fr/vps/definition/>
- [3] « Django documentation », Django Project. [En ligne]. Disponible sur:
<https://docs.djangoproject.com/en/5.1/>
- [4] « Poetry - Python dependency management and packaging made easy ». Consulté le: 25 mars 2025. [En ligne]. Disponible sur:
<https://python-poetry.org/>
- [5] « Wagtail 6.4.1 documentation ». [En ligne]. Disponible sur:
<https://docs.wagtail.org/en/stable/>
- [6] « Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. » [En ligne]. Disponible sur: <https://tailwindcss.com/>
- [7] « Theme variables - Core concepts ». [En ligne]. Disponible sur:
<https://tailwindcss.com/docs/theme>
- [8] « Documentation - OVHCloud ». [En ligne]. Disponible sur:
https://help.ovhcloud.com/csm?id=kb_home
- [9] « PostgreSQL: Documentation ». [En ligne]. Disponible sur:
<https://www.postgresql.org/docs/>